

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

2026.06.08

C++과 Java 차이 심화 개념정리

핵심	C++과 Java는 둘 다 객체지향을 지원하지만, 언어가 중요하게 보는 방향이 다르다. C++은 성능, 메모리 직접 제어, 하드웨어 가까운 개발에 강하고, Java는 JVM 기반 실행, 안정성, 유지보수, 웹 서버와 기업 시스템 개발에 강하다.
----	--

왜 이 주제가 중요한가

C++과 Java 차이를 배우는 이유는 단순히 "문법이 다르다"를 외우기 위해서가 아니다. 실제 개발에서는 분야에 따라 언어 선택이 달라진다.

예를 들어 게임 엔진, 로봇 제어, 자동차 제어 장치, IoT 센서처럼 하드웨어와 가깝고 빠른 반응이 중요한 곳은 C++이 많이 쓰인다. 반대로 웹 서버, 은행 시스템, 회사 업무 시스템처럼 안정적으로 오래 운영되고 유지보수가 중요한 곳은 Java가 많이 쓰인다.

즉 이 주제의 핵심 질문은 이것이다.

왜 어떤 상황에서는 C++을 쓰고, 왜 어떤 상황에서는 Java를 쓰는가? 위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

1. 가장 큰 차이 C++은 컴퓨터를 더 직접적으로 제어하는 언어이다.

Java는 JVM이라는 실행 환경 위에서 안정적으로 동작하도록 설계된 언어이다.

한 줄 비교:

C++ = 빠른 속도 + 직접 제어 + 높은 자유도 Java = 안정성 + 이식성 + 유지보수 + JVM 위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.
--

비유: C++은 수동 자동차와 비슷하다. 운전자가 기어, 속도, 조작을 직접 세밀하게 제어할 수 있다. 잘 다루면 빠르고 강력하지만 실수하면 위험하다.

Java는 자동 변속기와 안전장치가 많은 자동차와 비슷하다. 직접 제어는 줄어들지만 안정적이고 여러 사람이 운전하기 쉽다.

2. 실행 방식 차이 C++ 실행 흐름:

C++ 코드 -> 컴파일 -> 운영체제와 CPU에 맞는 실행 파일 -> 직접 실행 위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

예를 들어 Windows용으로 컴파일하면 '.exe' 파일이 만들어진다. 이 파일은 Windows 환경에 맞게 만들어진 것이므로 Linux에서 그대로 실행하기 어렵다. Linux에서 실행하려면 Linux용으로 다시 컴파일해야 한다.

Java 실행 흐름:

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

```
Java 코드
-> 컴파일
-> 바이트코드(.class)
-> JVM에서 실행
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 바로 운영체제용 실행 파일을 만드는 것이 아니라, JVM이 이해하는 바이트코드로 컴파일된다. 그 다음 JVM이 바이트코드를 실행한다.

왜 중요한가

C++은 실행 파일이 운영체제와 하드웨어에 더 직접적으로 맞춰져 있어 성능과 제어에 유리하다.

Java는 JVM만 있으면 여러 운영체제에서 같은 바이트코드를 실행하기 쉬워서 이식성에 유리하다.

공식 근거

Oracle의 JVM 명세는 JVM이 Java 플랫폼의 하드웨어와 운영체제 독립성을 담당하는 핵심 구성요소라고 설명한다. 또한 JVM은 특정 하드웨어나 운영체제를 가정하지 않고, '.class' 파일 형식의 바이트코드를 실행한다.

3. JVM이란 무엇인가 JVM은 Java Virtual Machine의 약자이다.

한국어로는 자바 가상 머신이라고 한다.

JVM은 Java 프로그램을 실행해주는 가상의 컴퓨터 같은 역할을 한다.

JVM의 역할:

- Java 바이트코드를 실행한다.
- 운영체제마다 다른 부분을 중간에서 맞춰준다.
- 메모리를 관리한다.
- 프로그램 실행 중 자주 쓰이는 코드를 최적화할 수 있다.
- 보안 검사를 통해 위험한 코드 실행을 줄인다.

흐름

```
Windows JVM -> Windows에서 Java 실행
Linux JVM -> Linux에서 Java 실행
macOS JVM -> macOS에서 Java 실행
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java 프로그램 입장에서는 JVM 위에서 실행되므로 운영체제 차이를 직접 신경 쓰는 부분이 줄어든다.

비유: JVM은 통역사와 비슷하다. Java 프로그램은 바이트코드라는 공통 언어로 말하고, JVM은 그것을 Windows, Linux, macOS 환경에 맞게 실행해준다.

발표용 문장: Java가 여러 운영체제에서 실행되기 쉬운 이유는 Java 코드가 바로 운영체제용 실행 파일이 되는 것이 아니라, 바이트코드로 변환된 뒤 JVM 위에서 실행되기 때문이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

4. 메모리 관리 차이 C++은 개발자가 메모리를 직접 관리할 수 있다.

```
int* p = new int;
*p = 10;
delete p;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

`new`로 메모리를 할당하고, `delete`로 직접 해제한다.

Java는 개발자가 객체를 만들 수는 있지만, C++처럼 `delete`로 직접 해제하지 않는다.

```
Person p = new Person();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java에서는 Garbage Collector, 즉 GC가 사용하지 않는 객체를 자동으로 정리한다.

왜 중요한가

C++은 메모리를 직접 제어할 수 있어 성능 최적화와 하드웨어 가까운 개발에 유리하다. 하지만 개발자가 해제를 깜빡하면 메모리 누수가 생기고, 이미 해제한 메모리를 다시 접근하면 오류가 생길 수 있다.

Java는 GC가 메모리를 자동으로 관리하기 때문에 개발자가 메모리 해제 실수를 덜 하게 된다. 대신 GC가 동작하는 시간에는 성능에 영향을 줄 수 있다.

공식 근거

Oracle의 Java GC 문서는 GC가 애플리케이션의 동적 메모리 할당 요청을 자동으로 관리한다고 설명한다. 또한 GC의 목적은 개발자가 직접 할당과 해제를 맞춰야 하는 부담을 줄이는 것이지만, 런타임 오버헤드가 생길 수 있다고 설명한다.

비교:

| 구분 | C++ | Java |

|---|---|---|

| 메모리 할당 | `new` | `new` || 메모리 해제 | `delete` 직접 사용 | GC가 자동 정리 || 장점 | 세밀한 제어, 성능 최적화 | 안전성, 개발 편의성 || 단점 | 메모리 누수 위험 | GC 오버헤드 가능 |

5. 포인터와 참조 차이 C++은 포인터를 직접 사용할 수 있다.

```
int a = 10;
int* p = &a;
cout << *p << endl;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

포인터는 메모리 주소를 저장하는 변수이다.

C++에서는 주소를 직접 다룰 수 있기 때문에 다음과 같은 작업이 가능하다.

- 동적 메모리 관리

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

- 배열과 메모리 직접 접근
- 하드웨어 가까운 제어
- 연결 리스트, 트리 같은 자료구조 구현

Java는 C++처럼 포인터 주소를 직접 조작하지 않는다.

```
Person p = new Person();
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java의 `p`는 객체를 가리키는 참조이지만, C++ 포인터처럼 주소 연산을 직접 할 수 없다.

왜 중요한가

포인터를 직접 다룰 수 있으면 강력하지만 위험하다. 잘못된 주소에 접근하면 프로그램이 비정상 종료되거나 보안 문제가 생길 수 있다.

Java는 주소 직접 조작을 막아 안전성을 높인다. 그래서 개발자는 객체를 참조해서 사용하지만 메모리 주소 자체를 계산하거나 이동시키지는 않는다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

공식 근거

Java 언어 명세는 Java 타입을 primitive type과 reference type으로 나누고, reference type의 값은 객체에 대한 reference라고 설명한다. 즉 Java는 객체를 참조로 다루지만 C++식 주소 조작 포인터를 언어 문법의 중심으로 제공하지 않는다.

6. 객체지향 구조 차이 C++은 절차지향과 객체지향을 모두 사용할 수 있다.

클래스 밖에 일반 함수를 만들 수 있다.

```
int add(int a, int b) {
    return a + b;
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

클래스를 사용해서 객체지향으로 작성할 수도 있다.

```
class Player {
public:
    void attack() {
        cout << "공격" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 대부분의 코드가 클래스 안에 들어간다.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 중요한가

C++은 자유도가 높다. 작은 프로그램은 함수 중심으로 작성하고, 큰 프로그램은 클래스로 설계할 수 있다.

Java는 클래스 중심 구조가 강하다. 그래서 여러 사람이 협업하는 프로젝트에서 코드 구조를 일정하게 유지하기 좋다.

비유: C++은 자유롭게 조립할 수 있는 공구 세트에 가깝다. Java는 정해진 규칙에 맞춰 부품을 조립하는 시스템에 가깝다.

7. 상속 차이 C++은 클래스 다중 상속을 지원한다.

```
class AttackSkill {
public:
    void attack() {}
};
```

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

```
class HealSkill {
public:
void heal() {}

};

class Paladin : public AttackSkill, public HealSkill {
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

‘Paladin’은 공격 기능과 회복 기능을 둘 다 물려받는다.

Java는 클래스 다중 상속을 허용하지 않는다.

```
class Student extends Person {
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java는 하나의 클래스만 상속받을 수 있다.

대신 여러 인터페이스를 구현할 수 있다.

```
class Paladin implements Attackable, Healable {
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

왜 중요한가

C++의 다중 상속은 여러 기능을 조합할 수 있어 강력하다. 하지만 부모 클래스들에 같은 이름의 변수나 함수가 있으면 충돌과 혼란이 생길 수 있다.

Java는 이런 복잡성을 줄이기 위해 클래스 다중 상속을 제한한다. 대신 인터페이스를 여러 개 구현하는 방식으로 타입의 다중성을 제공한다.

공식 근거

Oracle Java 튜토리얼은 Java가 여러 클래스를 상속하지 못하게 하는 이유 중 하나가 여러 부모 클래스의 상태나 구현을 동시에 상속할 때 생길 수 있는 충돌과 모호성을 피하기 위해서라고 설명한다. 또한 Java는 여러 인터페이스 구현을 통한 타입의 다중 상속을 지원한다.

8. 오버로딩과 오버라이딩 비교 C++과 Java 모두 오버로딩과 오버라이딩을 지원한다.

오버로딩:

```
void attack() {}
void attack(String target) {}
void attack(String target, int damage) {}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

같은 이름의 함수를 매개변수 다르게 여러 개 만드는 것이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

오버라이딩:

```
class Champion {
    void useSkill() {
        System.out.println("기본 스킬");
    }
}

class Garen extends Champion {
    @Override
    void useSkill() {
        System.out.println("가렌 스킬");
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

부모 클래스의 함수를 자식 클래스에서 다시 정의하는 것이다.

C++에서는 오버라이딩할 때 `virtual`이 중요하다.

```
class Champion {
public:
    virtual void useSkill() {
        cout << "기본 스킬" << endl;
    }
};
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java에서는 일반 인스턴스 메서드가 기본적으로 동적 바인딩된다. 그래서 C++처럼 매번 `virtual`을 붙이지 않는다.

왜 중요한가

이 차이는 게임 캐릭터 예시로 이해하기 쉽다.

```
Champion = 부모 클래스
Garen, Ahri = 자식 클래스
useSkill() = 공통 명령
실제 스킬 효과 = 자식마다 다름
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

C++에서는 부모 포인터로 자식 객체를 다룰 때 실제 자식 함수가 실행되게 하려면 부모 함수에 `virtual`을 붙인다. Java는 이런 객체지향 동작이 기본 메서드 호출에서 자연스럽게 일어난다.

9. 문법 차이 기본 제어문은 비슷하다.

C++:

```
for (int i = 0; i < 5; i++) {
    cout << i << endl;
}
```

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

```
}

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

Java:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

하지만 프로그램 기본 구조와 입출력은 다르다.

| 문법 요소 | C++ | Java |

|---|---|---|

| 출력 | `cout <<` | `System.out.println()` || 입력 | `cin >>` | `Scanner` 또는 `BufferedReader` || 문자열 | `string` | `String` || 참/거짓 | `bool` | `boolean` || 시작점 | `int main()` | `public static void main` || 일반 함수 | 클래스 밖 가능 | 클래스 안에 작성 || 객체 생성 | 스택 객체, 포인터 객체 가능 | 보통 `new`로 객체 생성 || 메모리 해제 | `delete` 가능 | 직접 `delete` 없음 |

10. 성능 차이 일반적으로 C++은 Java보다 성능을 더 직접적으로 제어하기 쉽다.

이유:

- 기계어 실행 파일로 컴파일된다.
- 메모리 배치와 해제를 직접 조절할 수 있다.
- 포인터와 참조를 사용해 저수준 제어가 가능하다.
- 불필요한 런타임 계층을 줄일 수 있다.

Java는 JVM 위에서 실행되기 때문에 구조상 런타임 계층이 있다. 하지만 JVM은 JIT 컴파일과 최적화를 통해 실행 중 성능을 높일 수 있다. 그래서 서버 프로그램에서는 Java도 충분히 빠르게 동작한다.

주의할 점

"C++은 항상 Java보다 빠르다"라고 말하면 위험하다. 프로그램 구조, 알고리즘, JVM 최적화, GC 설정, 하드웨어, 구현 방식에 따라 결과가 달라질 수 있다.

정확한 발표 문장:

```
C++은 성능과 메모리 제어를 직접 하기 쉬워 고성능 시스템에 유리한 경우가 많다.
Java는 JVM 최적화 덕분에 서버 환경에서도 충분히 좋은 성능을 낼 수 있지만,
GC와 JVM 런타임의 영향을 고려해야 한다.
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

11. IoT 분야에서 C++이 유리한 이유 IoT 기기는 보통 작고 제한적인 환경에서 동작한다.

특징:

- CPU 성능이 낮을 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

- 메모리가 작다.
- 전력 소모를 줄여야 한다.
- 센서, 모터, 통신 장치 같은 하드웨어를 직접 제어한다.
- 실시간 반응이 중요할 수 있다.

이런 환경에서는 C++이 유리하다.

이유:

- 메모리 사용량을 직접 관리할 수 있다.
- 하드웨어 가까운 코드 작성이 가능하다.
- 실행 속도를 세밀하게 최적화할 수 있다.
- 운영체제 없이 또는 작은 런타임 위에서도 사용할 수 있다.

예시:

- 온도 센서 값 읽기
- 모터 회전 제어
- 로봇 팔 제어
- 자동차 ECU
- 드론 비행 제어
- 라즈베리파이 또는 임베디드 보드 제어 프로그램

비유: IoT 기기는 작은 배터리와 작은 두뇌를 가진 장치이다. 이런 장치에서는 무거운 실행 환경보다 가볍고 직접 제어 가능한 코드가 유리하다.

12. 웹 분야에서 Java가 유리한 이유 웹 서버는 빠르기만 해서는 안 된다. 오래 안정적으로 돌아가야 하고, 여러 개발자가 유지보수할 수 있어야 한다.

Java가 웹에서 많이 쓰이는 이유:

- JVM 위에서 여러 서버 환경에 대응하기 쉽다.
- GC가 메모리를 자동 관리한다.
- Spring Framework, Spring Boot 같은 웹 개발 생태계가 강하다.
- 코드 구조가 클래스 중심이라 대규모 협업에 유리하다.
- 예외 처리, 타입 안정성, 패키지 구조가 체계적이다.
- 기업 시스템에서 오랫동안 사용되어 사례와 자료가 많다.

공식 근거

Spring 공식 문서는 Spring Web MVC가 Servlet API 기반의 원래 웹 프레임워크이며 Spring Framework 초기부터 포함되어 있었다고 설명한다. 즉 Java 웹 서버 생태계에서 Spring MVC는 오래된 핵심 기술 중 하나이다.

예시:

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

- 쇼핑몰 상품/주문 서버
- 은행 계좌 시스템
- 회사 인사 관리 시스템
- 예약 시스템
- 공공기관 업무 시스템
- Spring 기반 REST API 서버

비유: 웹 서버는 24시간 열려 있는 편의점과 비슷하다. 혼자 빠르게 움직이는 것보다, 교대 근무자들이 규칙을 지키며 안정적으로 운영할 수 있는 구조가 중요하다. Java는 이런 장기 운영과 협업에 강하다.

13. Android와 Java Android 개발에서는 과거부터 Java가 많이 사용되었다. 현재 Android 공식 문서는 Kotlin을 권장하지만, Java를 아는 개발자를 위한 자료와 상호 운용 문서도 제공한다.

정리

- 예전 Android 앱은 Java 기반 코드가 많다.
- 현재는 Kotlin이 권장되지만 Java 지식은 Android 이해에 여전히 도움이 된다.
- Java 문법과 객체지향 개념을 알면 Android의 클래스, 인터페이스, 상속 구조를 이해하기 쉽다.

발표에서는 이렇게 말하면 안전하다.

Android는 현재 Kotlin 중심으로 발전하고 있지만,
기존 Android 생태계에는 Java 코드가 많고 Java 객체지향 개념을 이해하는 것이 도움이 된다.

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

14. 게임 비유로 보는 C++과 Java C++은 게임 엔진 내부를 만드는 느낌이다.

예:

- 캐릭터의 물리 충돌 계산
- 프레임마다 그래픽 렌더링
- 메모리 풀 관리
- 실시간 입력 처리

이런 부분은 성능과 제어가 중요해서 C++이 잘 맞는다.

Java는 게임 운영 서버를 만드는 느낌이다.

예:

- 로그인 서버
- 유저 정보 관리
- 결제/상점 서버
- 매칭 서버

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

- 랭킹 시스템

이런 부분은 안정성, 네트워크 처리, 데이터 관리, 유지보수가 중요해서 Java가 잘 맞을 수 있다.

정리 정리

게임 화면을 빠르게 그리는 엔진 쪽 = C++이 유리
게임 유저 정보를 안정적으로 관리하는 서버 쪽 = Java가 유리
위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

15. 상황별 선택 기준 다음 질문을 던지면 어떤 언어가 적합한지 판단하기 쉽다.

성능이 가장 중요한가?

그렇다면 C++을 고려한다.

하드웨어를 직접 제어해야 하는가?

그렇다면 C++이 유리하다.

메모리 사용을 직접 조절해야 하는가? 그렇다면 C++이 유리하다.

여러 운영체제와 서버 환경에서 안정적으로 실행되어야 하는가? 그렇다면 Java가 유리하다.

대규모 웹 서버나 기업 시스템인가? 그렇다면 Java가 유리하다.

개발자가 직접 메모리 해제를 관리하기 어렵거나 안정성이 중요한가? 그렇다면 Java가 유리하다.

16. 비교표

| 구분 | C++ | Java |

|---|---|---|

| 실행 방식 | 기계어 실행 파일로 컴파일 | 바이트코드로 컴파일 후 JVM 실행 || 핵심 장점 | 성능, 직접 제어 | 안정성, 이식성, 유지보수 || 메모리 관리 | 직접 관리 가능 | GC 자동 관리 || 포인터 | 직접 사용 가능 | 직접 포인터 조작 없음 || 객체지향 | 절차지향 + 객체지향 혼합 가능 | 클래스 중심 구조 || 다중 상속 | 클래스 다중 상속 가능 | 클래스 다중 상속 불가, 인터페이스 다중 구현 || 성능 | 저수준 최적화 유리 | JVM/JIT 최적화, 서버 성능 우수 || 주요 분야 | 게임 엔진, 임베디드, IoT, 시스템 | 웹 서버, 기업 시스템, Android 기준 코드 || 위험 요소 | 메모리 오류, 포인터 오류 | GC 오버헤드, JVM 필요 || 비유 | 수동 자동차 | 안전장치 많은 자동 자동차 |

17. 발표 예상 질문과 답변 질문: Java는 JVM 때문에 무조건 느린가?

답변: 무조건 느리다고 할 수 없다. Java는 JVM 위에서 실행되지만 JIT 컴파일과 최적화를 사용한다. 서버 프로그램에서는 Java도 충분히 빠르다. 다만 하드웨어 직접 제어나 극단적인 성능 최적화가 필요한 경우에는 C++이 유리할 수 있다.

질문: C++이 Java보다 항상 좋은가?

답변: 아니다. C++은 성능과 제어가 강하지만 메모리 관리 실수 위험이 있다. Java는 직접 제어는 적지만 안정성과 유지보수, 웹 서버 개발에 강하다.

질문: Java에는 포인터가 없는가?

subject	title	check ☐ ☐ ☐ ☐ ☐
수요일스터디	수요일스터디 - C++과 Java 차이 개념정리	date

답변: Java는 객체를 참조로 다루지만 C++처럼 메모리 주소를 직접 조작하는 포인터 문법은 제공하지 않는다. 그래서 안정성은 높아지지만 저수준 제어는 어렵다.

질문: IoT에서는 Java를 아예 안 쓰는가?

답변: 아니다. 아주 작은 장치 내부 제어는 C/C++이 유리하지만, IoT 데이터를 모으는 서버나 게이트웨이, 관리 시스템에서는 Java를 사용할 수 있다.

질문: 웹 서버는 C++로 만들 수 없는가?

답변: 만들 수 있다. 하지만 Java는 Spring 같은 프레임워크와 기업용 생태계가 강해서 대규모 웹 서버 개발과 유지보수에 많이 사용된다.

18. 최종 정리 C++과 Java의 차이는 단순한 문법 차이가 아니라 설계 철학의 차이이다.

C++은 빠른 실행, 메모리 직접 제어, 하드웨어 가까운 개발에 강하다. 그래서 게임 엔진, 임베디드, IoT 장치 내부 제어, 고성능 시스템에 잘 맞는다.

Java는 JVM 기반 실행, 자동 메모리 관리, 클래스 중심 구조, 강력한 웹 생태계가 장점이다. 그래서 웹 서버, 기업 시스템, 유지보수가 중요한 대규모 서비스에 잘 맞는다.

발표용 핵심 문장:

C++은 기계를 더 직접적으로 다루기 위한 언어이고,
Java는 여러 환경에서 안정적으로 오래 운영되는 프로그램을 만들기 좋은 언어이다.
위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

참고 자료

- Oracle, Java Virtual Machine Specification: JVM은 Java 플랫폼의 하드웨어/운영체제 독립성을 담당하고 `.class` 파일의 바이트코드를 실행한다고 설명한다.
- Oracle, Java Garbage Collection Tuning: GC는 동적 메모리를 자동으로 관리하며, 개발자의 직접 메모리 해제 부담을 줄이지만 런타임 오버헤드가 있을 수 있다고 설명한다.
- Oracle Java Tutorial, Multiple Inheritance: Java가 클래스 다중 상속을 제한하는 이유와 인터페이스 다중 구현을 설명한다.
- Spring Framework Reference, Spring Web MVC: Spring Web MVC가 Servlet API 기반 웹 프레임워크이며 Spring Framework 초기부터 포함된 기술이라고 설명한다.
- Android Developers, Kotlin and Android: 현재 Android는 Kotlin을 권장하지만 Java 개발자를 위한 자료와 상호 운용 흐름도 제공한다.